

Einfluss von Architektur- und Trainingsentscheidungen auf die Performance von CNNs zur Human Activity Recognition

Lisa Weickenmeier
Hochschule Bielefeld

`lisa.weickenmeier@hsbi.de`

Abstract

In diesem Projekt wird das Training verschiedener Convolutional Neural Networks (CNNs) mittels Keras untersucht. Ziel war es, die Auswirkungen unterschiedlicher Modellarchitekturen auf die Klassifizierungsgenauigkeit zu vergleichen. Hierzu wurden mehrere Architekturen mit variierenden Tiefen, Layer-Kombinationen und Hyperparametern implementiert und trainiert. Der als Grundlage verwendete Datensatz enthält Bilder von Menschen, die verschiedenen Aktivitäten nachgehen, welche erkannt werden sollen. Die Resultate zeigen deutliche Unterschiede in der Modellperformance in Abhängigkeit von der Architektur, insbesondere hinsichtlich Overfitting und Generalisierung. Die Analyse verdeutlicht, welche architektonischen Anpassungen signifikante Verbesserungen bewirken können. Abschließend werden Empfehlungen für den sinnvollen Einsatz spezifischer Layer-Konfigurationen abgeleitet.

1. Einleitung

Die automatische Erkennung menschlicher Aktivitäten (Human Activity Recognition, HAR) auf Basis von Bilddaten spielt in zahlreichen Anwendungsbereichen eine wichtige Rolle. Beispiele sind Videoüberwachung und Sicherheit, Sport- und Bewegungsanalyse, Assistenzsysteme im häuslichen Umfeld oder die Mensch-Maschine-Interaktion. Für solche Aufgaben werden heute überwiegend Convolutional Neural Networks (CNNs) eingesetzt, da sie relevante visuelle Merkmale direkt aus den Eingabebildern lernen können und klassische, manuell entworfene Merkmalsrepräsentationen in vielen Szenarien deutlich übertreffen. Ihre Leistungsfähigkeit hängt jedoch stark von der gewählten Architektur und den verwendeten Trainingsstrategien ab.

Eine zentrale Herausforderung besteht darin, ein geeignetes Gleichgewicht zwischen Modellkomplexität und Generalisierungsfähigkeit zu finden. Zu kleine Modelle können die zugrunde liegenden Muster der Daten nicht

adäquat erfassen (*Underfitting*), während zu große Modelle dazu neigen, sich zu stark an die Trainingsdaten anzupassen und auf Validierungs- und Testdaten schlechter abzuschneiden (*Overfitting*). In der Praxis ist es daher notwendig, architektonische Entscheidungen und Trainingsstrategien systematisch zu untersuchen, anstatt lediglich einzelne, isolierte Modifikationen vorzunehmen.

Das Ziel dieses Projekts ist es, verschiedene CNN-Architekturen und Trainingsstrategien für die Klassifikation menschlicher Aktivitäten in Bildern vergleichend zu analysieren. Dazu werden insgesamt 19 Modellvarianten betrachtet, die sich unter anderem in der Verwendung von Eingangsnormalisierung, Datenaugmentierung, Aktivierungsfunktionen, Pooling-Strategien, *BatchNormalization*, *Dropout* sowie in ihrer Gesamtgröße und Tiefe unterscheiden. Auf Basis der experimentellen Ergebnisse sollen Empfehlungen für geeignete Layer-Konfigurationen und Trainingsverfahren für den verwendeten HAR-Datensatz abgeleitet werden.

Der weitere Aufbau dieses Papers gestaltet sich wie folgt: Abschnitt 2 beschreibt zunächst den verwendeten Datensatz, die durchgeführten Vorverarbeitungsschritte sowie die untersuchten Modellarchitekturen und die gemeinsame Trainingskonfiguration. In Abschnitt 3 werden die quantitativen Ergebnisse vorgestellt und anhand von Lernkurven erläutert. Abschnitt 4 diskutiert diese Ergebnisse im Hinblick auf Overfitting, Generalisierung und den Einfluss einzelner Designentscheidungen. Abschließend fasst Abschnitt 5 die wichtigsten Erkenntnisse zusammen und gibt einen Ausblick auf mögliche weiterführende Arbeiten.

2. Eigene Methoden

In diesem Abschnitt werden die im Projekt verwendeten Daten, die vorgenommenen Vorverarbeitungsschritte sowie die entwickelten Modellarchitekturen und Trainingskonfigurationen beschrieben, um die experimentelle Vorgehensweise nachvollziehbar darzustellen.

2.1. Datensatz

Als Datengrundlage wurde ein aufbereiteter *Human Activity Recognition* (HAR) Datensatz verwendet, in dem auch die Testdaten gelabelt sind [2]. Dieser Datensatz basiert auf einer CNN-Challenge aus dem Jahr 2022 [1]. Ursprünglich umfasst der Datensatz 15 Aktivitätsklassen, die jeweils einem ganzzahligen Label im Bereich von 0 bis 14 zugeordnet sind:

- | | |
|-----------------|------------------------|
| 0. sitting | 8. calling |
| 1. using_laptop | 9. laughing |
| 2. hugging | 10. eating |
| 3. sleeping | 11. fighting |
| 4. drinking | 12. listening_to_music |
| 5. clapping | 13. running |
| 6. dancing | 14. texting |
| 7. cycling | |

Aufgrund der starken visuellen Ähnlichkeit einiger Klassen und der darin enthaltenen Bilder wurden die Klassen *sitting* (0), *drinking* (4), *laughing* (9), *running* (13) und *texting* (14) aus dem Datensatz entfernt. Anschließend verblieben 10 Klassen.

Der Trainingsdatensatz umfasste insgesamt 8400 Bilder (840 Bilder je Klasse). Von diesen wurden 20 % für die Validierung abgetrennt.

Der Testdatensatz enthielt weitere 3564 Bilder, worin jede Klasse enthalten war.

Vor dem Training wurden alle Bilder einer Vorverarbeitung unterzogen. Die dabei verwendeten Schritte werden im folgenden Abschnitt beschrieben.

2.2. Datenvorverarbeitung und Augmentierung

Die ursprünglichen Bilder aus dem Datensatz konnten aufgrund unterschiedlicher Auflösungen nicht direkt für das Training der Modelle verwendet werden. Daher wurden sie beim Laden mittels *Resize* auf eine einheitliche Größe von 224×224 Pixel gebracht. Wie im vorigen Abschnitt beschrieben, wurden 20 % der Trainingsbilder als Validierungsmenge abgetrennt. Die Batch-Größe für das Training wurde auf 32 festgelegt und im Verlauf des Projekts nicht verändert, um den Fokus auf die Auswirkungen der Änderungen an der Modellarchitektur zu legen, die in Abschnitt 2.3 erläutert werden.

Neben dem *Resize*, das stets direkt beim Laden der Daten ausgeführt wurde, kamen sukzessive zwei weitere Formen der Vorverarbeitung hinzu: Normalisierung und Datenaugmentierung.

Die Normalisierung der Farbwerte auf den Wertebereich $[0, 1]$ wurde ab dem zweiten Modell ergänzt. Dabei wird jeder RGB-Kanal separat durch 255 skaliert, sodass die drei Kanäle erhalten bleiben und lediglich ihre Wertebereiche angepasst werden.

Ab Modell 3 wurde zusätzlich eine Datenaugmentierung eingesetzt. Hierfür wurden die Keras-Layer *RandomFlip*, *RandomRotation*, *RandomZoom* und *RandomContrast* verwendet. Die Faktoren für *RandomRotation*, *RandomZoom* und *RandomContrast* wurden zunächst konservativ gewählt (0,05, 0,05 bzw. 0,1) und in einem späteren, deutlich größeren Modell erhöht, um eine stärkere Variation der Trainingsdaten zu erzielen.

RandomFlip wurde anfangs sowohl horizontal als auch vertikal angewendet. Ab Modell 15 wurde das vertikale Spiegeln deaktiviert, sodass nur noch horizontale Spiegelungen verwendet wurden. Hintergrund ist, dass in dem Datensatz menschliche Aktivitäten dargestellt werden und ein vertikal gespiegeltes Bild (Kopf nach unten) eine unplausible und im realen Einsatz unwahrscheinliche Situation darstellt. Es wurde daher angenommen, dass auch die Testbilder sowie zukünftige Eingabebilder in aufrechter Orientierung vorliegen.

2.3. Modellarchitekturen

Im Rahmen des Projekts wurden insgesamt 19 Modellvarianten trainiert und miteinander verglichen. Ausgangspunkt war ein mit Hilfe eines Large-Language-Modells entworfenes CNN, das bereits ein relativ gutes Ergebnis erzielte. Dieses Zielmodell enthielt viele der später getesteten Schichten und Hyperparameter-Anpassungen und diente als grobe Orientierung.

Darauf aufbauend wurde zunächst ein Basismodell definiert, das nur die notwendigen Minimum-Schichten enthält. Tabelle 1 zeigt dessen Architektur.

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (conv2d)	(None, 224, 224, 32)	896
conv2d_1 (conv2d)	(None, 224, 224, 32)	9,248
conv2d_2 (conv2d)	(None, 216, 216, 32)	9,248
conv2d_3 (conv2d)	(None, 216, 216, 32)	9,248
max_pooling2d (maxpooling2d)	(None, 108, 108, 32)	0
conv2d_4 (conv2d)	(None, 108, 108, 32)	9,248
max_pooling2d_1 (maxpooling2d)	(None, 54, 54, 32)	0
flatten (Flatten)	(None, 89888)	0
dense (dense)	(None, 128)	11,585,792
dense_1 (dense)	(None, 10)	1,290
Total params: 11,594,976 (44.04 MB)		
Trainable params: 11,594,976 (44.04 MB)		
Non-trainable params: 0 (0.00 B)		

Figure 1. Architektur des Keras-Basismodells

Auf dieser Basis wurden die Architekturen schrittweise verändert. In den meisten Fällen wurde pro Modell nur eine einzelne Modifikation vorgenommen, um deren Effekt möglichst eindeutig zuordnen zu können.

Wie in Abschnitt 2.2 beschrieben, kamen in den frühen Modellvarianten zunächst Normalisierung und Datenaugmentierung hinzu. Mit dem augmentierten Modell wurden außerdem ein frühzeitiger Trainingsabbruch (*EarlyStop*-

ping) sowie eine dynamische Anpassung der Lernrate (*ReduceLROnPlateau*) erprobt. Aufgrund der positiven Ergebnisse wurden diese beiden Mechanismen für alle weiteren Modelle beibehalten.

Neben Änderungen an Anzahl oder Art der Schichten wurden teilweise ausschließlich Schichtparameter variiert. So kamen in Modell 4 und Modell 5 alternative Aktivierungsfunktionen (ELU bzw. Swish) zum Einsatz. In Modell 9 wurde das Padding in allen *Conv2D*-Schichten auf *same* statt *valid* gesetzt. Im anschließenden Modell 10 wurde die Standardinitialisierung *glorot-uniform* in allen ReLU-Schichten durch *he-normal* ersetzt. Von diesen Parameteranpassungen wurde lediglich das geänderte Padding in den späteren Varianten weiterverwendet.

Zusätzlich zu den Schichten des Basismodells wurden in verschiedenen Varianten *BatchNormalization*-, *Dropout*- und *GlobalAveragePooling2D*-Schichten integriert. *BatchNormalization* und *GlobalAveragePooling2D* wurden dabei bewusst mehrfach ein- und wieder ausgebaut, um ihre Wirkung in kleineren und größeren Netzen zu untersuchen. Ab Modell 11 wurde die Gesamtgröße der Architektur sukzessive erhöht, indem weitere *Conv2D*-Blöcke mit mehr Feature Maps ergänzt wurden. Tabelle 1 fasst die jeweils zentrale Änderung pro Modell zusammen.

Modell	Änderung
Modell 1	Basis-Modell
Modell 2	+ Normalisierung (Input)
Modell 3	+ Augmentation (Input)
Modell 4	ReLU → ELU
Modell 5	ELU → Swish
Modell 6	BatchNorm nach Conv2D
Modell 7	BatchNorm raus, Dropout nach Dense
Modell 8	Flatten → GlobalAvgPool
Modell 9	GlobalAvgPool → Flatten, Padding= <i>same</i>
Modell 10	<i>he-normal</i> Init statt <i>glorot-uniform</i>
Modell 11	mehr Feature Maps, neue Conv2D-Struktur
Modell 12	+ Conv2D-Block (128 Maps)
Modell 13	+ Dense Layer
Modell 14	Flatten → GlobalAvgPool
Modell 15	RandomFlip nur horizontal
Modell 16	+ BatchNorm
Modell 17	+ Dropout nach Dense
Modell 18	+ Conv2D-Block (256 Maps)
Modell 19	größeres Netz, stärkere Augmentation

Table 1. Übersicht der Hauptänderungen der Modellvarianten

Im nächsten Abschnitt werden die Ergebnisse der einzelnen Änderungen betrachten und damit auch die Gründe, wieso manchen Änderungen beibehalten wurden und andere nicht.

Im Abschnitt 3 werden die Ergebnisse dieser Architekturänderungen vorgestellt und anschließend darauf

basierend begründet, welche Modifikationen in späteren Modellen beibehalten wurden und welche nicht.

2.4. Trainingskonfiguration

Unabhängig von der jeweiligen Modellarchitektur teilen sich alle Modelle eine Reihe gemeinsamer Hyperparameter. Das *Resize* der Eingabebilder, die Batch-Größe sowie die Aufteilung in Trainings- und Validierungsmenge sind in Abschnitt 2.2 beschrieben. Darüber hinaus wurden alle Modelle mit dem Optimierer *Adam* und der Verlustfunktion *Sparse Categorical Crossentropy* trainiert. Die maximale Anzahl an Trainingsepochen lag bei 100. Durch den Einsatz von *EarlyStopping* ab Modell 3 wurde dieser Wert jedoch häufig nicht ausgeschöpft.

Als Abbruchkriterium für *EarlyStopping* diene stets die Validierungsgenauigkeit. Sobald sich diese über 12 aufeinanderfolgende Epochen nicht mehr verbessert, wird das Training beendet. Anschließend werden die Gewichte der Epoche mit der höchsten Validierungsgenauigkeit geladen und im Modell gesichert.

Für die dynamische Anpassung der Lernrate (*ReduceLROnPlateau*) wurde der Validierungsfehler (*val_loss*) als Kriterium herangezogen. Bleibt dieser über drei Epochen hinweg unverändert beziehungsweise verbessert sich nicht, wird die Lernrate halbiert. Als untere Schranke wurde eine minimale Lernrate von $1 \cdot 10^{-6}$ festgelegt.

Der vollständige Quellcode sowie die Skripte zur Reproduktion der Experimente sind im zugehörigen Git-Repository öffentlich verfügbar [3].

3. Ergebnisse

3.1. Quantitative Ergebnisse

Im folgenden werden zunächst die quantitativen Ergebnisse anhand der Trainings- und Testgenauigkeit betrachtet. Die Werte der einzelnen Modelle sind in Tabelle 2 zusammengefasst. Modell 3 ist dort dreimal aufgeführt, da in drei aufeinanderfolgenden Trainingsläufen zunächst nur die Datenaugmentierung, anschließend zusätzlich *EarlyStopping* und schließlich *ReduceLROnPlateau* aktiviert wurden.

Die ersten drei Modellvarianten illustrieren sehr deutlich den Effekt von Normalisierung, Augmentierung und Trainingsstrategien. Während das Basismodell (Modell 1) und das Modell mit zusätzlicher Eingangsnormalisierung (Modell 2) jeweils eine Trainingsgenauigkeit von 100 % erreichen, liegt die Testgenauigkeit bei lediglich 23,34 % bzw. 30,11 % und zeigt damit starkes Overfitting. Durch Datenaugmentierung sowie den Einsatz von *EarlyStopping* und *ReduceLROnPlateau* (Modell 3**) sinkt die Trainingsgenauigkeit auf 68,81 %, während die Testgenauigkeit auf 41,44 % steigt. Die Modelle werden also weniger stark an die Trainingsdaten angepasst, generalisieren aber deutlich besser.

Modell	Trainingsgenauigkeit	Testgenauigkeit
Modell 1	100,00%	23,34%
Modell 2	100,00%	30,11%
Modell 3	92,90%	38,08%
	74,60%*	40,80%*
	68,81%**	41,44%**
Modell 4	9,84%	11,17%
Modell 5	74,75%	38,97%
Modell 6	10,27%	11,64%
Modell 7	56,07%	41,86%
Modell 8	41,15%	38,24%
Modell 9	51,40%	41,22%
Modell 10	57,47%	46,32%
Modell 11	56,47%	44,42%
Modell 12	57,56%	44,25%
Modell 13	62,16%	40,54%
Modell 14	57,35%	48,99%
Modell 15	67,49%	53,76%
Modell 16	76,80%	57,72%
Modell 17	70,10%	55,33%
Modell 18	82,95%	63,50%
Modell 19	68,71%	57,63%

Table 2. Trainings- und Testgenauigkeit der Modellvarianten. (*): mit *EarlyStopping*; (**): mit *EarlyStopping* und *ReduceLROnPlateau*.

Die Varianten mit veränderten Aktivierungsfunktionen (Modell 4 und Modell 5) sowie umfangreicher *BatchNormalization* (Modell 6) erzielen keine Verbesserung der Testgenauigkeit gegenüber Modell 3** und liegen teilweise sogar deutlich darunter. Erst ab Modell 7, bei dem *BatchNormalization* entfernt und stattdessen *Dropout* eingeführt wird, erreicht die Testgenauigkeit mit 41,86 % wieder das Niveau von Modell 3**, bei gleichzeitig moderater Trainingsgenauigkeit.

Einen weiteren Sprung nach oben zeigt Modell 10. Hier wird durch vorab geändertes Padding und eine angepasste Initialisierung eine Testgenauigkeit von 46,32 % erreicht, bei vergleichsweise geringer Differenz zwischen Trainings- und Testgenauigkeit. In den darauf folgenden Modellen 11 bis 13 bleiben die Werte weitgehend stabil, teils mit leichtem Overfitting (höhere Trainingsgenauigkeit bei rückläufiger Testgenauigkeit).

Ab Modell 14 ist ein kontinuierlicher Anstieg der Testgenauigkeit zu beobachten. Die Kombination aus *GlobalAveragePooling2D*, gezielter Datenaugmentierung und dem schrittweisen Ausbau der Architektur führt bis Modell 18 zu einer Testgenauigkeit von 63,50 %, dem besten Wert aller untersuchten Modelle. Gleichzeitig steigt die Trainingsgenauigkeit auf 82,95 %, was auf zunehmendes Overfitting hindeutet. Modell 19 mit weiter verstärkter Augmentierung erreicht eine etwas niedrigere Testgenauigkeit von 57,63 %

und liegt damit hinter Modell 18 zurück, allerdings mit deutlich geringerem Overfitting.

Insgesamt zeigt sich, dass reine Architekturvergrößerung ohne geeignete Regularisierung zu starkem Overfitting führt, während eine Kombination aus Datenaugmentierung, *EarlyStopping*, *Dropout* und gezielt vergrößerter Modellkapazität eine deutliche Verbesserung der Testgenauigkeit ermöglicht.

3.2. Lernkurven

Neben den Endwerten von Trainings- und Testgenauigkeit liefern die Lernkurven zusätzliche Einblicke in das Verhalten der Modelle während des Trainings. Die folgenden Abbildungen zeigen exemplarisch die Verläufe von Trainings- und Validierungsgenauigkeit für das Basismodell (Modell 1) sowie das beste Modell (Modell 18). Für alle Diagramme wurde die Trainingsgenauigkeit in orange und die Validierungsgenauigkeit in hellblau dargestellt.

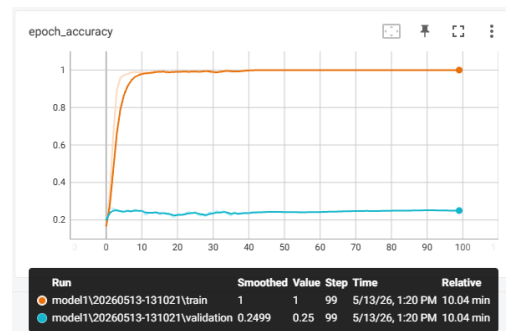


Figure 2. Trainings- und Validierungsgenauigkeit von Modell 1 über die Epochen hinweg

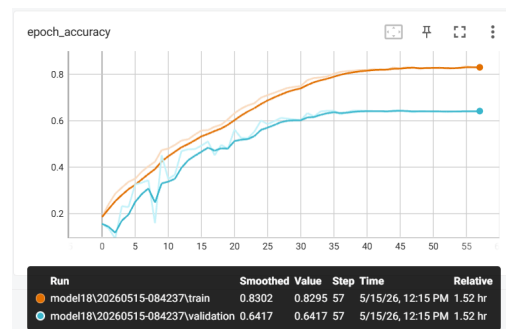


Figure 3. Trainings- und Validierungsgenauigkeit von Modell 18 über die Epochen hinweg

In Abbildung 2 ist zu erkennen, dass die Trainingsgenauigkeit von Modell 1 sehr schnell auf nahezu 100 % ansteigt, während die Validierungsgenauigkeit bereits nach wenigen Epochen stagniert. Zwischen beiden Kurven öffnet sich früh eine deutliche Lücke, was das in Abschnitt 3 beschriebene starke Overfitting bestätigt. Die Net-

zarchitektur ist in der Lage, die Trainingsdaten praktisch auswendig zu lernen, kann dieses Wissen jedoch kaum auf die Validierungs- bzw. Testdaten übertragen.

Demgegenüber zeigt Abbildung 3 die Lernkurve von Model 18. Hier steigen Trainings- und Validierungsgenauigkeit relativ gleichmäßig an, wobei die Validierungskurve etwas unterhalb der Trainingskurve verläuft. Die Lücke zwischen beiden Verläufen ist deutlich kleiner als bei Model 1, was auf eine bessere Generalisierung hinweist. Zugleich ist erkennbar, dass die Validierungsgenauigkeit über mehrere Epochen hinweg zunimmt und erst später plateauförmig verläuft. *EarlyStopping* beendet das Training nach 57 Epochen in einem Bereich, in dem sich die Validierungsgenauigkeit nicht mehr verbessert, sodass das Modell nicht unnötig weiter in Richtung Overfitting getrieben wird.

4. Diskussion

Im Folgenden werden die Ergebnisse verallgemeinert und es wird diskutiert, welche Architekturempfehlungen sich aus den Experimenten ableiten lassen und inwiefern diese auf andere Anwendungsfälle übertragbar sind.

4.1. Overfitting und Generalisierung

Die frühen Modellvarianten sind deutlich von Overfitting geprägt. Model 1 verfügt mit etwa 11,5 Millionen Parametern über eine deutlich höhere Kapazität, als es der verfügbare Datensatz ohne zusätzliche Maßnahmen sinnvoll zulässt. Die Trainingsgenauigkeit erreicht zwar 100 %, die Testgenauigkeit bleibt mit 23,34 % jedoch auf einem sehr niedrigen Niveau.

Durch die schrittweise eingeführte Datenaugmentierung wurde der effektive Umfang des Trainingsdatensatzes erhöht und die Variabilität innerhalb der Klassen vergrößert. Die positive Auswirkung lässt sich insbesondere in Model 3 beobachten: Die Trainingsgenauigkeit sinkt gegenüber Model 2, während die Testgenauigkeit deutlich ansteigt. Zusätzliche, leicht veränderte Ansichten derselben Aktivitäten helfen dem Modell, robustere Merkmalsrepräsentationen zu lernen, anstatt sich zu stark an die konkreten Trainingsbeispiele anzupassen.

Eine weitere Reduktion des Overfittings wird durch den Einsatz von *Dropout* erreicht. Der Effekt zeigt sich besonders im Vergleich von Model 3** zu Model 7, in dem lediglich eine zusätzliche Dropout-Schicht im klassifizierenden Teil des Netzes ergänzt wurde. Obwohl sich die Testgenauigkeit nur moderat verändert, verringert sich die Differenz zwischen Trainings- und Testgenauigkeit von 27,37 % auf 14,21 %. Dies deutet darauf hin, dass ein wesentlicher Teil des Overfittings in den vollvernetzten Schichten am Ende des Netzes auftrat. Die *Conv2D*-Schichten extrahieren sinnvolle Merkmale, die anschließenden *Dense*-Schichten passen sich jedoch zu stark an spezi-

fische Kombinationen dieser Merkmale an. Durch Dropout wird diese Spezialisierung abgeschwächt, sodass das Modell allgemeinere Entscheidungsregeln lernt.

4.2. Einfluss einzelner Architektur-Elemente

Neben der Reduktion von Overfitting lag ein zentrales Ziel darin, die absolute Testgenauigkeit durch gezielte architektonische Anpassungen zu erhöhen.

Aktivierungsfunktionen. Neben der standardmäßig eingesetzten Aktivierungsfunktion *ReLU* wurden mit *ELU* (Model 4) und *Swish* (Model 5) zwei alternative Varianten getestet. Beim Einsatz von ELU zeigte sich kein stabiler Trainingsverlauf: Die Trainingsgenauigkeit schwankte stark zwischen den Epochen und blieb insgesamt auf einem sehr niedrigen Niveau. Ein möglicher Grund dafür ist eine ungünstige Kombination aus Lernrate und Aktivierungsfunktion, sodass das Optimierungsverfahren kein sinnvolles Minimum findet. Mit Swish war ein reguläres Training möglich, die erzielte Testgenauigkeit lag jedoch unter der von ReLU, bei gleichzeitig stärkerem Overfitting. Unter den in diesem Projekt verwendeten Einstellungen bietet ReLU somit den besten Kompromiss aus Stabilität und Generalisierungsfähigkeit. Ein erneuter Vergleich von Swish und ReLU in einem noch größeren Modell (z.B. auf Basis von Model 18 oder Model 19) könnte interessant sein.

Batch Normalization. Zusätzlich zur Dropout-Schicht im klassifizierenden Teil des Netzes wurde *BatchNormalization* in mehreren Modellen eingesetzt (u.a. Model 6 und Model 16). Der Vergleich dieser Varianten mit ihren jeweiligen Vorgängermodellen macht deutlich, dass der Effekt stark von der Modellgröße abhängt. In einem relativ kleinen Netz wie Model 6, das nur wenige Feature Maps pro *Conv2D*-Schicht extrahiert, führt das Hinzufügen von Batch-Normalisierung zu einem Einbruch der Trainings- und Testgenauigkeit. Eine plausible Erklärung ist, dass bei geringer Modellkapazität die Normalisierung die ohnehin begrenzte Vielfalt der Merkmalsrepräsentationen zu stark einschränkt, sodass für die anschließende Klassifikation zu wenig aussagekräftige Features übrigbleiben.

In größeren Architekturen wie Model 16 kann Batch-Normalisierung dagegen helfen, die Verteilung der Zwischenausgaben zu stabilisieren und Ausreißer in den Aktivierungen abzuschwächen. In Kombination mit ausreichender Modellkapazität und Datenaugmentierung trägt sie dort zu höheren Testgenauigkeiten bei. Insgesamt legt dies nahe, dass Batch-Normalisierung in sehr kleinen Netzen vorsichtig eingesetzt werden sollte, während sie in tieferen Modellen mit vielen Feature Maps den Trainingsprozess unterstützen kann.

GlobalAveragePooling2D. Ein ähnliches kapazitätsabhängiges Verhalten zeigt sich beim Einsatz von *GlobalAveragePooling2D*. Im kleineren Model 8 führt der Austausch von *Flatten* durch *GlobalAveragePooling2D* zwar zu einer stärkeren Generalisierung (geringeres Overfitting), gleichzeitig sinkt die Testgenauigkeit leicht. Durch die starke Reduktion der räumlichen Informationen stehen den *Dense*-Schichten weniger frei verfügbare Parameter zur Verfügung, was in dieser Modellgröße offenbar zu einer leichten Unteranpassung führt.

Im größeren Model 14 wirkt sich *GlobalAveragePooling2D* deutlich positiver aus. Das Overfitting nimmt ab, während die Testgenauigkeit deutlich steigt. Hier scheint die Kombination aus größerer Feature-Extraktion in den *Conv2D*-Schichten und kompakter Klassifikation günstiger zu sein als eine große, vollständig vernetzte Schicht nach *Flatten*. Dies spricht dafür, *GlobalAveragePooling2D* insbesondere in größeren Netzen als Standardstrategie im Übergang von *Conv2D*- zu *Dense*-Schichten zu verwenden.

Modellgröße. Schließlich zeigt die sukzessive Vergrößerung der Modelle, dass eine höhere Kapazität prinzipiell zu deutlich besseren Ergebnissen führen kann. Model 18 mit einem zusätzlichen *Conv2D*-Block und insgesamt mehr Feature Maps erreicht die höchste Testgenauigkeit aller Varianten, weist jedoch auch wieder eine erhöhte Lücke zwischen Trainings- und Testgenauigkeit auf. Model 19 vergrößert die Architektur und die Augmentierung nochmals, ohne die Testgenauigkeit weiter zu steigern. Dies deutet darauf hin, dass für den betrachteten Datensatz ein Bereich erreicht ist, in dem zusätzliche Parameter nicht mehr zu besserer Generalisierung führen, sondern vor allem das Risiko von Overfitting und Instabilitäten im Training erhöhen.

5. Zusammenfassung

In dieser Arbeit wurde untersucht, wie sich unterschiedliche Architektur- und Trainingsentscheidungen auf die Leistung von Convolutional Neural Networks zur bildbasierten Erkennung menschlicher Aktivitäten auswirken. Ausgehend von einem einfachen Basismodell wurden insgesamt 19 Modellvarianten entwickelt, in denen schrittweise Eingangsnormalisierung, Datenaugmentierung, alternative Aktivierungsfunktionen, verschiedene Pooling-Strategien, *BatchNormalization*, *Dropout* sowie eine zunehmende Modellgröße und -tiefe erprobt wurden.

Die Ergebnisse zeigen, dass eine reine Erweiterung der Architektur ohne geeignete Regularisierung zu starkem Overfitting führt. Frühe Modelle erreichen zwar eine Trainingsgenauigkeit von 100 %, bleiben in der Testgenauigkeit jedoch deutlich hinter den Erwartungen zurück. Erst die Kombination aus Datenaugmentierung, *EarlyStopping*, *Dropout* und einem gezielten, nicht übertriebenen

Ausbau der Modellkapazität führt zu einer deutlichen Verbesserung der Generalisierung. Das beste untersuchte Modell (Model 18) erreicht eine Testgenauigkeit von 63,50 % und bildet damit einen sinnvollen Kompromiss zwischen Kapazität und Overfitting.

Für den verwendeten HAR-Datensatz lassen sich daraus mehrere praktische Empfehlungen ableiten. Eine frühe Normalisierung der Eingabebilder und eine maßvolle Datenaugmentierung (insbesondere horizontales Spiegeln und leichte geometrische bzw. kontrastbezogene Transformationen) erweisen sich als robuste Basis. *Dropout* nach den *Dense*-Schichten trägt wesentlich dazu bei, Overfitting in der Klassifikation zu reduzieren. *BatchNormalization* sollte in kleineren Netzen nur zurückhaltend eingesetzt werden, kann in tieferen Architekturen mit vielen Feature Maps jedoch zur Stabilisierung des Trainings beitragen. Im Übergang von *Conv2D*- zu *Dense*-Schichten bietet sich bei größeren Modellen *GlobalAveragePooling2D* als sinnvolle Alternative zu *Flatten* an, da es die Anzahl der Parameter reduziert und die Generalisierung unterstützt.

Für zukünftige Arbeiten bietet sich insbesondere eine systematische Hyperparameter-Optimierung an, beispielsweise im Hinblick auf Lernraten, Dropout-Raten und die Gewichtsinitialisierung. Darüber hinaus könnten weitere Regularisierungstechniken wie L2-Gewichtsregularisierung oder *Label Smoothing* untersucht werden.

References

- [1] Data sprint 76: Human activity recognition. <https://aiplanet.com/challenges/233/data-sprint-76-human-activity-recognition-233/overview/about>, 2026. Zugriff am 16.05.2026. 2
- [2] Jiashuo Wang. Har - human activity recognition dataset with label. <https://www.kaggle.com/datasets/wangboluo/harhuman-activity-recognition-dataset-with-label>, 2022. Zugriff am 16.05.2026. 2
- [3] Lisa Weickenmeier. Code und modelle zur CNN-basierten Human Activity Recognition. <https://github.com/LisaWckn/deeplearning>, 2026. Zugriff: 18.05.2026. 3

A. Anhang: Zusatzmaterial

Die folgenden Bilder enthalten Diagramme über die Genauigkeit und den Fehler der einzelnen Modelle. Die Trainingsdaten sind dabei stets in oranger Farbe und die Validierungsdaten in hellblauer Farbe abgebildet.

Zudem sind die Werte der jeweils letzten Epoche ersichtlich.

A.1. Diagramme zu Modell 1

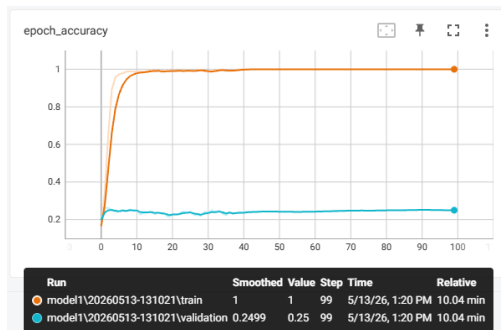


Figure 4. Trainings- und Validierungsgenauigkeit von Modell 1 über die Epochen hinweg

A.2. Diagramme zu Modell 2

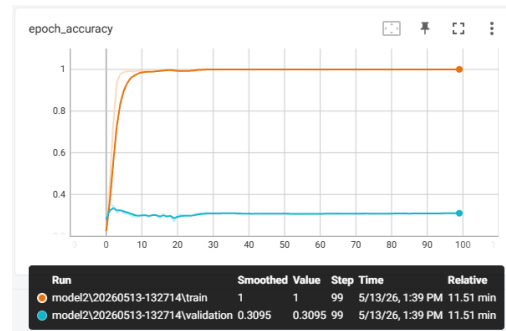


Figure 6. Trainings- und Validierungsgenauigkeit von Modell 2 über die Epochen hinweg

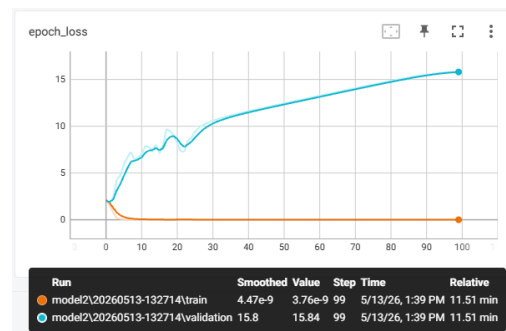


Figure 7. Trainings- und Validierungsfehler von Modell 2 über die Epochen hinweg

A.3. Diagramme zu Modell 3

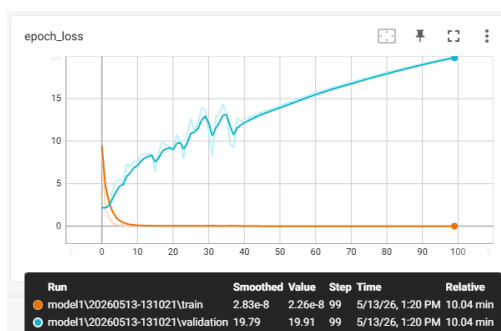


Figure 5. Trainings- und Validierungsfehler von Modell 1 über die Epochen hinweg

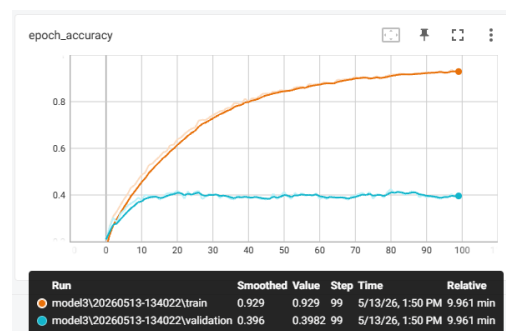


Figure 8. Trainings- und Validierungsgenauigkeit von Modell 3 über die Epochen hinweg

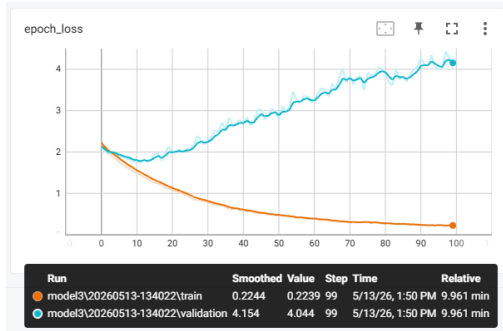


Figure 9. Trainings- und Validierungsfehler von Modell 3 über die Epochen hinweg

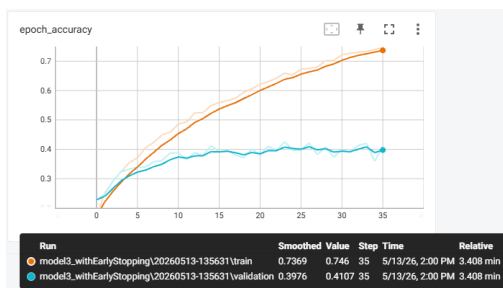


Figure 10. Trainings- und Validierungsgenauigkeit von Modell 3 mit EarlyStopping über die Epochen hinweg

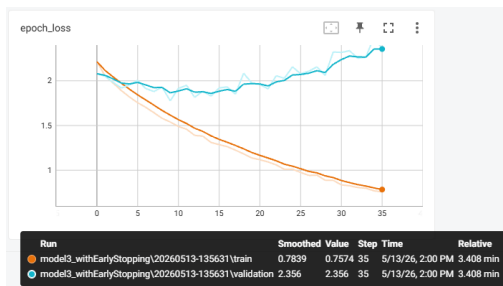


Figure 11. Trainings- und Validierungsfehler von Modell 3 mit EarlyStopping über die Epochen hinweg

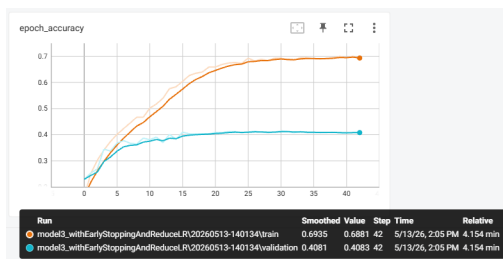


Figure 12. Trainings- und Validierungsgenauigkeit von Modell 3 mit EarlyStopping und ReduceLRonPlateau über die Epochen hinweg

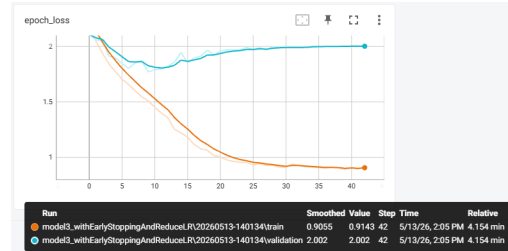


Figure 13. Trainings- und Validierungsfehler von Modell 3 mit EarlyStopping und ReduceLRonPlateau über die Epochen hinweg

A.4. Diagramme zu Modell 4

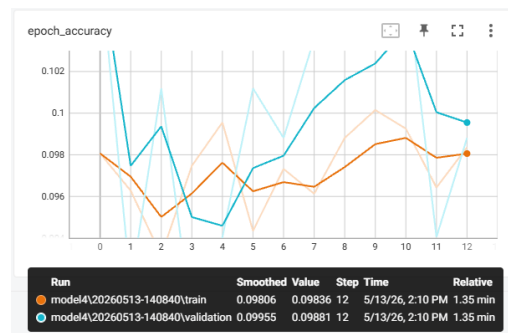


Figure 14. Trainings- und Validierungsgenauigkeit von Modell 4 über die Epochen hinweg

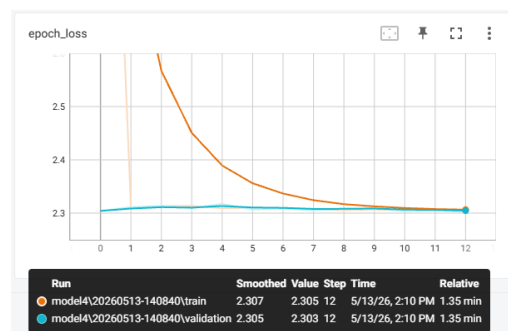


Figure 15. Trainings- und Validierungsfehler von Modell 4 über die Epochen hinweg

A.5. Diagramme zu Modell 5

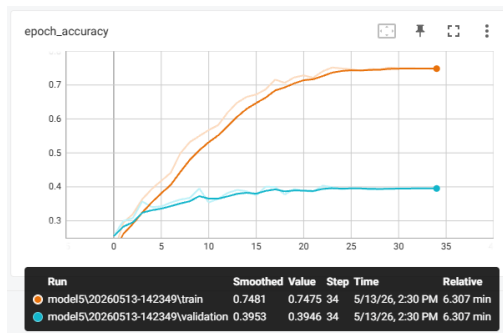


Figure 16. Trainings- und Validierungsgenauigkeit von Modell 5 über die Epochen hinweg

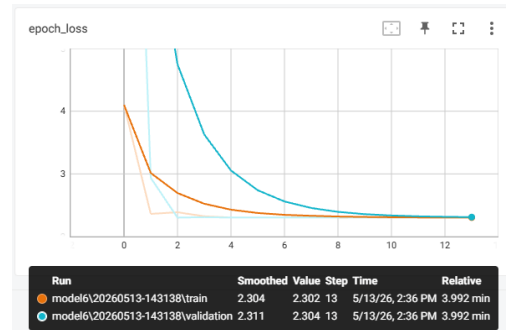


Figure 19. Trainings- und Validierungsfehler von Modell 6 über die Epochen hinweg

A.7. Diagramme zu Modell 7

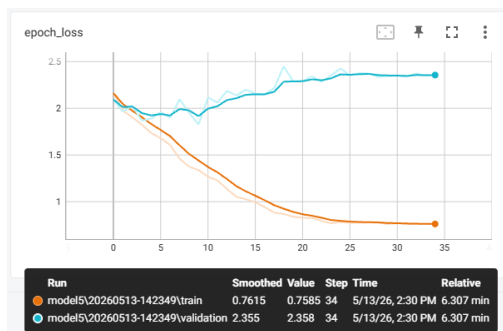


Figure 17. Trainings- und Validierungsfehler von Modell 5 über die Epochen hinweg

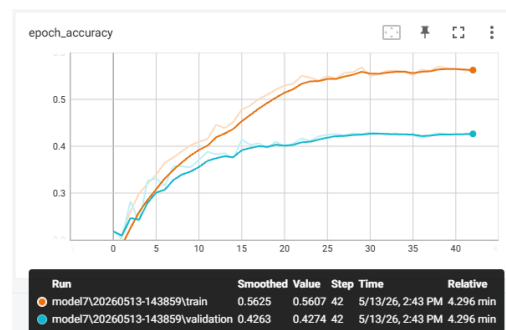


Figure 20. Trainings- und Validierungsgenauigkeit von Modell 7 über die Epochen hinweg

A.6. Diagramme zu Modell 6

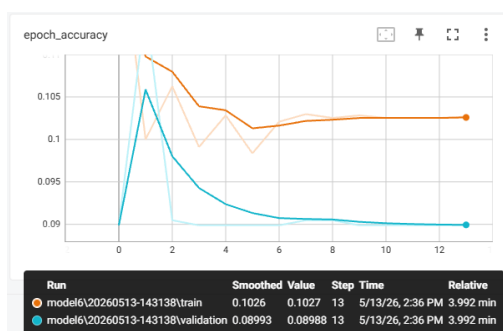


Figure 18. Trainings- und Validierungsgenauigkeit von Modell 6 über die Epochen hinweg

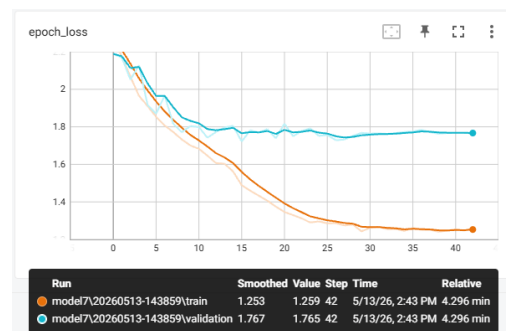


Figure 21. Trainings- und Validierungsfehler von Modell 7 über die Epochen hinweg

A.8. Diagramme zu Modell 8

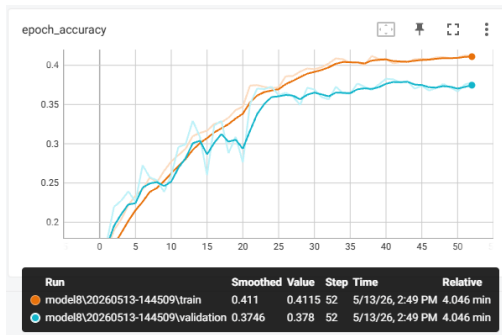


Figure 22. Trainings- und Validierungsgenauigkeit von Modell 8 über die Epochen hinweg

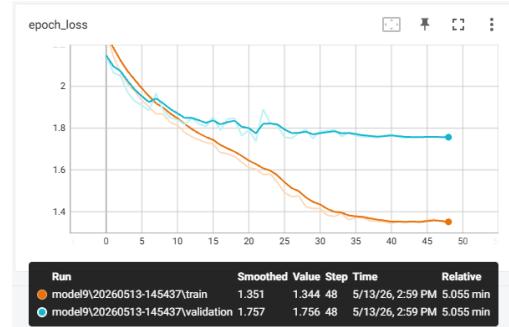


Figure 25. Trainings- und Validierungsfehler von Modell 9 über die Epochen hinweg

A.10. Diagramme zu Modell 10

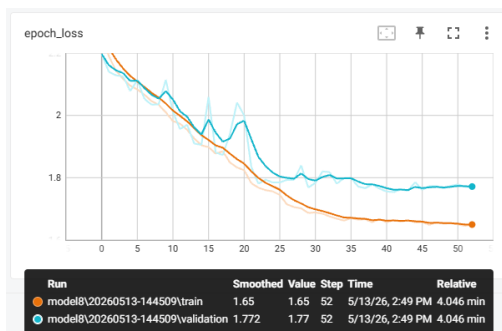


Figure 23. Trainings- und Validierungsfehler von Modell 8 über die Epochen hinweg

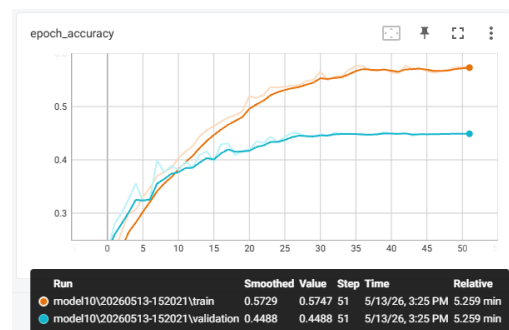


Figure 26. Trainings- und Validierungsgenauigkeit von Modell 10 über die Epochen hinweg

A.9. Diagramme zu Modell 9

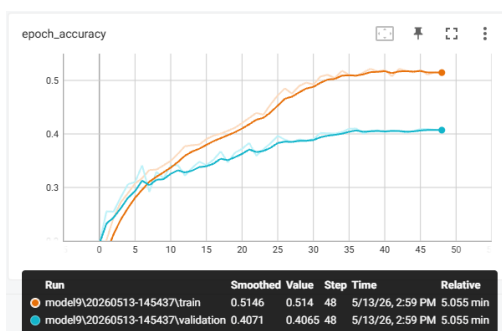


Figure 24. Trainings- und Validierungsgenauigkeit von Modell 9 über die Epochen hinweg

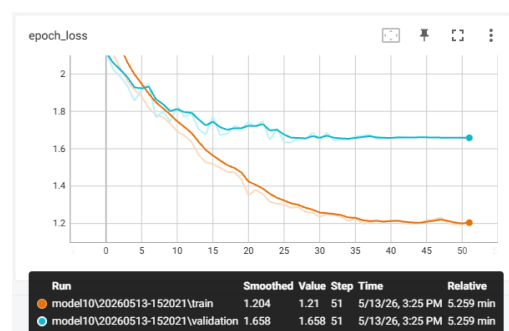


Figure 27. Trainings- und Validierungsfehler von Modell 10 über die Epochen hinweg

A.11. Diagramme zu Modell 11

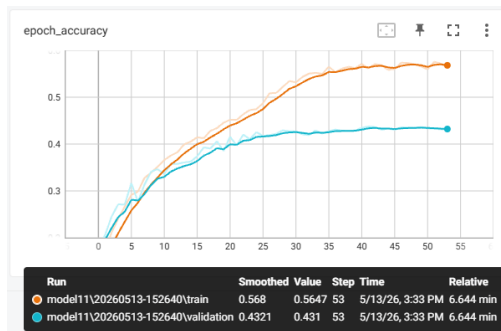


Figure 28. Trainings- und Validierungsgenauigkeit von Modell 11 über die Epochen hinweg

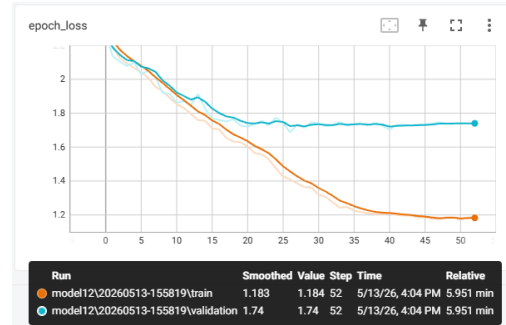


Figure 31. Trainings- und Validierungsfehler von Modell 12 über die Epochen hinweg

A.13. Diagramme zu Modell 13

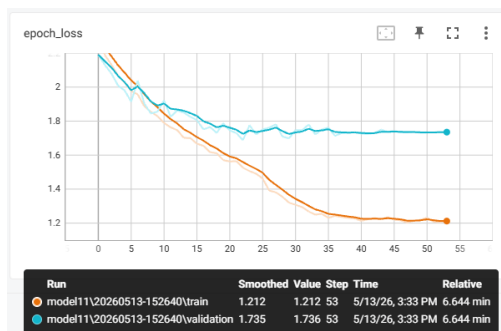


Figure 29. Trainings- und Validierungsfehler von Modell 11 über die Epochen hinweg

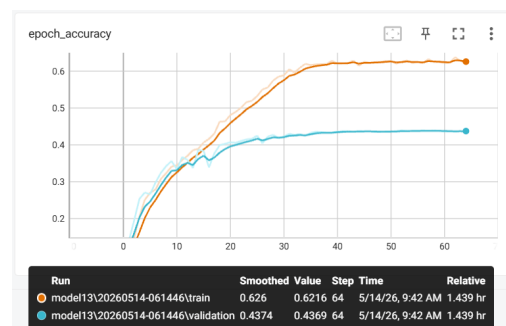


Figure 32. Trainings- und Validierungsgenauigkeit von Modell 13 über die Epochen hinweg

A.12. Diagramme zu Modell 12

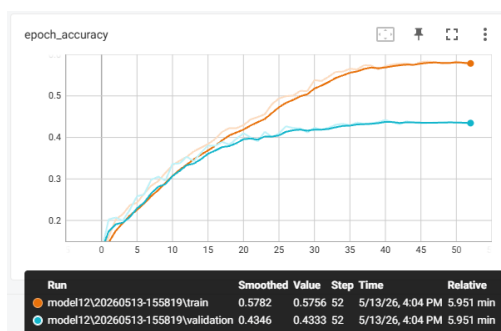


Figure 30. Trainings- und Validierungsgenauigkeit von Modell 12 über die Epochen hinweg

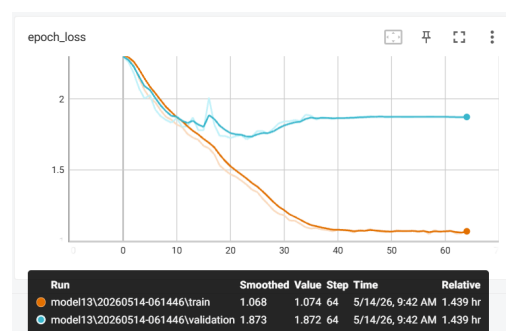


Figure 33. Trainings- und Validierungsfehler von Modell 13 über die Epochen hinweg

A.14. Diagramme zu Modell 14

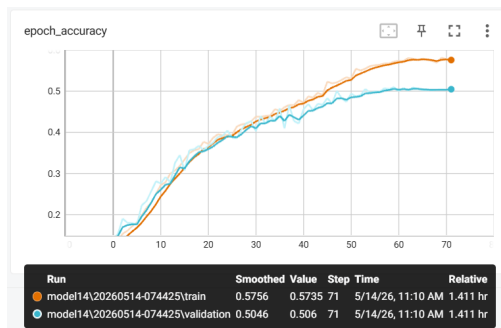


Figure 34. Trainings- und Validierungsgenauigkeit von Modell 14 über die Epochen hinweg

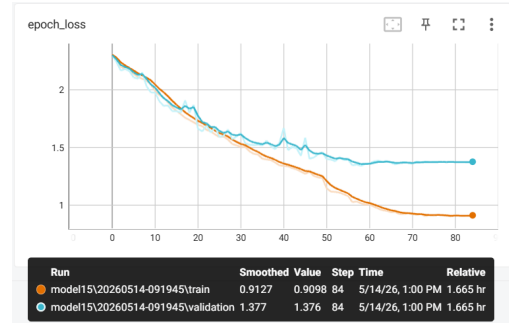


Figure 37. Trainings- und Validierungsfehler von Modell 15 über die Epochen hinweg

A.16. Diagramme zu Modell 16

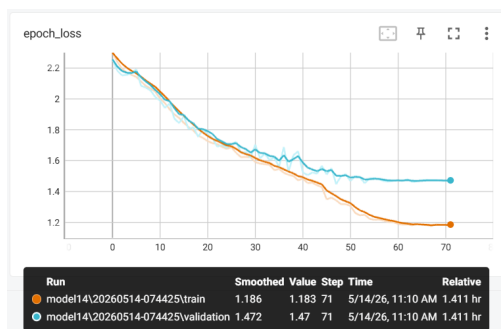


Figure 35. Trainings- und Validierungsfehler von Modell 14 über die Epochen hinweg

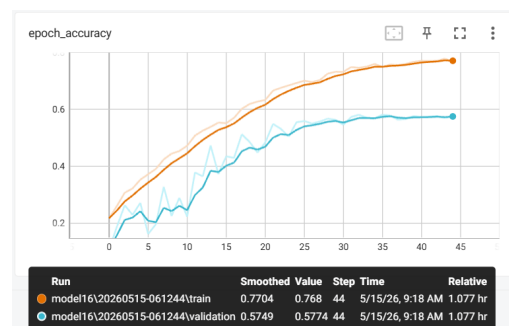


Figure 38. Trainings- und Validierungsgenauigkeit von Modell 16 über die Epochen hinweg

A.15. Diagramme zu Modell 15

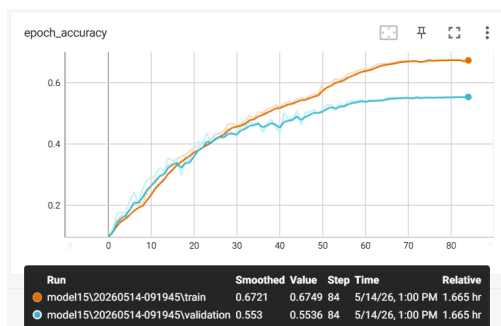


Figure 36. Trainings- und Validierungsgenauigkeit von Modell 15 über die Epochen hinweg

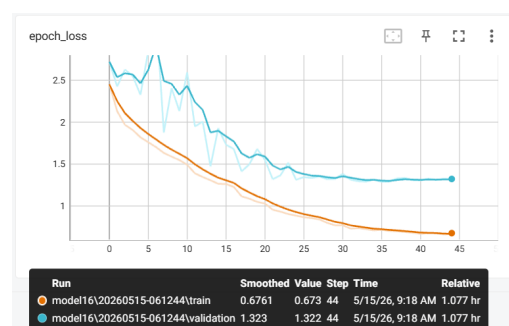


Figure 39. Trainings- und Validierungsfehler von Modell 16 über die Epochen hinweg

A.17. Diagramme zu Modell 17

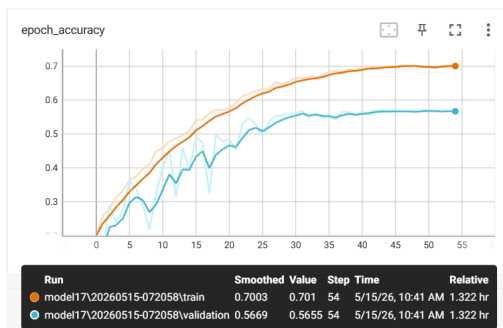


Figure 40. Trainings- und Validierungsgenauigkeit von Modell 17 über die Epochen hinweg

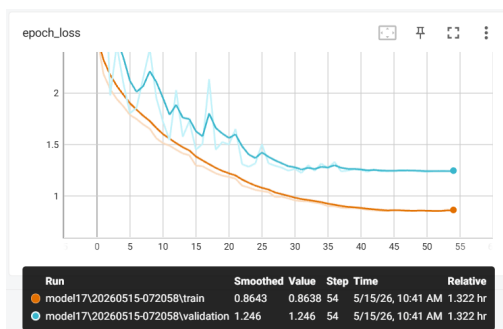


Figure 41. Trainings- und Validierungsfehler von Modell 17 über die Epochen hinweg

A.18. Diagramme zu Modell 18

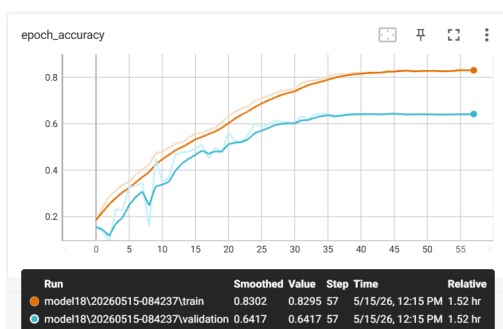


Figure 42. Trainings- und Validierungsgenauigkeit von Modell 18 über die Epochen hinweg

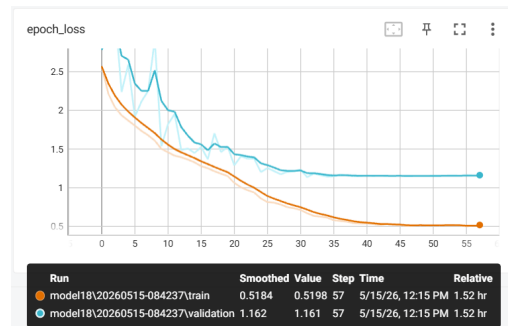


Figure 43. Trainings- und Validierungsfehler von Modell 18 über die Epochen hinweg

A.19. Diagramme zu Modell 19

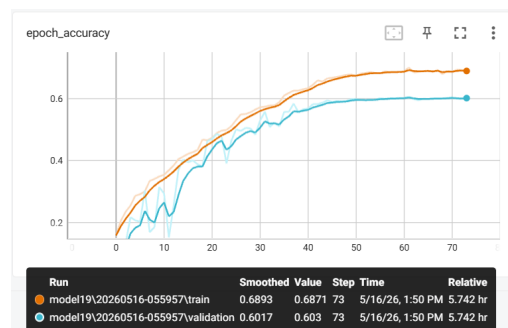


Figure 44. Trainings- und Validierungsgenauigkeit von Modell 19 über die Epochen hinweg

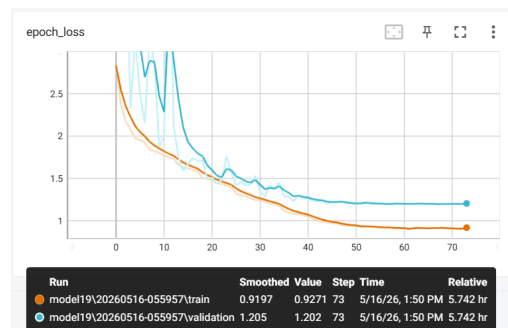


Figure 45. Trainings- und Validierungsfehler von Modell 19 über die Epochen hinweg